SAM RIVERA

Senior Full Stack Engineer Cedar Falls, Example State

sam.example.com | github.com/sam-rivera-example | 202-555-0142 | sam.rivera@example.com

SUMMARY

Experience spans frontend interfaces, backend APIs, relational data and maintenance of established software. This fictional resume exercises document generation and validation without representing a real applicant.

TECHNICAL SKILLS

Languages: TypeScript, JavaScript, Python

Web and data: React, Node.js, PostgreSQL

PROFESSIONAL EXPERIENCE

Northwind Labs

2020–2026

Senior Software Engineer

- Built a React and TypeScript scheduling interface for a fictional operations team, with Node.js APIs and PostgreSQL records. Kept existing reports usable while adding the new workflow.
- Introduced version-checked writes and explicit conflict messages so concurrent edits could not silently replace a newer saved record. Added integration tests against a temporary database.
- Replaced an aging data import pipeline in small steps, retaining source snapshots and checking counts before publication. Documented rollback steps for the operators who maintained the service.
- Traced intermittent request failures through proxy and database logs, then reproduced the connection lifecycle in an isolated test before changing timeout behavior.
- Maintained shared form controls with keyboard navigation and accessible labels. Reviewed changes with screenshots and behavior tests rather than relying on visual similarity alone.
- Wrote a short upgrade plan covering existing workflows, constraints and staged acceptance. Used review feedback to resolve the highest-impact behavior differences before rollout.

SELECTED PROJECTS

Bluepine Software

2017–2020

Software Engineer

- Designed a fictional inventory ledger with immutable receipts, revision checks and bounded retry rules. Verified interrupted writes and concurrent readers through isolated database tests.
- Maintained scheduled imports, deployment checks and application diagnostics for a synthetic reporting service. Kept configuration separate from runtime state and documented recovery paths.
- Bound generated documents to captured resume and posting hashes so a later source revision could not be mistaken for the evidence used by an earlier artifact.
- Built keyboard-operable selection and filtering controls with clear focus states, meaningful empty states and recoverable errors. Kept form answers stable across local view changes.
- Validated external payloads at the HTTP boundary and passed typed records into rule functions. Kept network retries independent from non-repeatable publication actions.
- Used real temporary stores and selectable PDFs to test persistence and document integrity. Synthetic transports covered orchestration without making claims about live provider behavior.
- Reviewed small changes by running the affected workflows, checking failure cases and recording unresolved limitations. Kept changes reversible and separated organization work from behavior changes.
- Kept imported research, personal workflow metadata and immutable generated packages in separate storage areas. Reimports preserved user choices and never rewrote submitted documents.
- Added bounded activity records for retries and failures while avoiding repeated success noise. Exposed progress using actual item counts instead of a simulated percentage.
- Read the existing service and its history before replacing behavior. Wrote a concise continuation note with the relevant source paths, commands and verification results.

EDUCATION

B.S. Computer Science, Cedar Valley University, 2017